Table des matières

Exercices corrigés 07 — Charger en incrémental 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 4
Mises en situation 4
Corrigé 6

Exercices corrigés 07 — Charger en incrémental

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur ce que vous venez de faire au labo ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 07 et du Labo 07.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Le chantier et le régime permanent. Nommez les trois questions que le régime permanent pose et que le chantier ne posait pas, et dites laquelle est la plus importante.
2. Les deux régimes. Dites ce que dbt fait d'un modèle incrémental à la première exécution et aux suivantes, et ce qui décide laquelle des deux s'applique.
3. Les trois pièces. Nommez `is_incremental()`, `{{ this }}` et `unique_key`, et donnez le rôle de chacune en une phrase.
4. Vitesse et justesse. Dites laquelle des trois pièces sert la vitesse et laquelle sert la justesse, puis dites laquelle des deux fautes est la grave.
5. `{{ this }}`. Dites ce qu'il désigne, et pourquoi il ne s'emploie qu'à l'intérieur du bloc ouvert par `is_incremental()`.
6. La stratégie. Dites ce que `delete+insert` fait, et ce qu'elle suppose pour être juste.
7. Quoi rendre incrémental. Donnez la règle, puis classez les six faits de Boréal selon qu'ils la satisfont ou non.
8. [labo] La perte du filtre naïf. Donnez le nombre de sorties perdues, ce qu'elles emportent, et les quatre jours où elles tombent.

© 2026 iAlexMG. Tous droits réservés.

9. Pourquoi elle n’est pas aléatoire. Dites pourquoi la perte frappe toujours la fin de la période, et ce que cela implique pour un rapport mensuel.
10. La cadence. Dites ce que la même faute coûterait sur toute l’archive au rythme mensuel et au rythme quotidien, et énoncez la règle en une phrase.
11. La bonne colonne. Dites sur quelle colonne le filtre doit porter, pourquoi ce n’est jamais celle que le métier lit, et ce que cela impose de garder dans le fait.
12. La fenêtre de reprise. Dites ce qu’elle assure, ce qu’elle coûte sur les deux dernières passes du labo, et pourquoi elle ne rapporte rien sur ce jeu de données.
13. La largeur. Dites d’où vient la largeur de trois jours, et ce qu’une fenêtre de trente jours dirait de celui qui l’a écrite.
14. [labo] La propriété vérifiée. Donnez les deux mesures qui établissent qu’un message n’arrive jamais avec une date de réception antérieure à une date déjà vue.
15. La clé et la relecture. Dites ce qui arrive à une fenêtre de reprise sans `unique_key`, et donnez le chiffre mesuré au labo.
16. `--full-refresh`. Dites quand on le lance, et ce que les deux commandes qui le mesurent rendent sur un instantané.
17. Le contrôle qui compte. Dites quel contrôle vérifie vraiment un modèle incrémental, et pourquoi aucun test de grain ni de relation ne le remplace.
18. [labo] Le fait d’accumulation. Dites pourquoi `fct_reclamations` reste en `table`, donnez le chiffre qui le justifie, et dites pourquoi une réconciliation sur l’argent ne verrait rien.
19. [labo] La dérive de schéma. Dites ce que le micrologiciel 2.0 change, donnez les deux comptes, et dites où les deux noms se réunissent et pourquoi là.

1. Le semi-structuré. Nommez les trois difficultés de la télémétrie, et dites ce que chacune devient au `STAGING`.
2. Les deux horloges. Dites ce que `recu_le` et `charge_le` mesurent, et laquelle porte le filtre du fait.
3. Deux environnements. Dites ce qui décide où un modèle atterrit, et ce que le code en sait.
4. L’orchestration. Dites ce que `dbt build` fournit déjà, ce qu’un ordonnanceur ajoute, et à quelle formation appartient le reste.
5. [labo] Le verdict et sa contre-épreuve. Donnez les deux taux de réclamation de refroidissement, puis dites ce que les trois codes qui ne séparent pas les groupes apportent au raisonnement.

Questions pratiques

P1. [labo] Le journal des passes. Écrivez la requête qui donne, par mois de réception, le nombre de sorties, le nombre de véhicules distincts et les kilomètres, depuis octobre 2025. Donnez le résultat et dites ce que la dernière ligne apprend sur la saison.

© 2026 iAlexMG. Tous droits réservés.

P2. Le prix de la fenêtre. Écrivez la requête qui dit, sans lancer dbt, combien de lignes la prochaine passe relirait pour une fenêtre de 1, 3, 7 et 30 jours. Donnez le résultat et dites ce que vous en concluez sur le choix de la largeur.

P3. [labo] Le contrôle en test. Le labo compare `stg_sorties` et `fct_sorties` à la main. Écrivez le test singulier qui le fait à chaque construction, dites ce qu'il coûte, et dites pourquoi on le garde malgré ce coût.

P4. Le retard, par tranche. Écrivez la requête qui range les sorties en quatre tranches de retard, moins d'une heure, une à vingt-quatre heures, vingt-quatre à soixante-douze heures, et plus. Donnez le résultat et dites ce qu'il faudrait changer si une cinquième tranche apparaissait.

P5. (avancé) Un fait sans horloge d'arrivée. On vous demande de rendre `fct_enregistrements` incrémental. Sa source ne porte pas de `recu_le`. Écrivez le bloc incrémental qui convient, en nommant la colonne et en disant ce que vous devez ajouter au `STAGING` pour l'obtenir, puis dites pourquoi le lot de livraison ne ferait pas l'affaire.

P6. Les codes par gamme. Écrivez la requête qui donne, par gamme de véhicules, le nombre de véhicules ayant levé un code en roulant et le nombre de codes levés. Donnez le résultat et dites ce qui frappe.

P7. Le parc connecté. Écrivez la requête qui donne, par année-modèle, les véhicules assemblés, ceux vendus au détail et ceux qui publient de la télémétrie. Donnez le résultat et dites quelle précaution il impose à toute analyse fondée sur la télémétrie.

P8. (avancé) La fenêtre retrouvée. Sans lui donner les dates de la dérive, écrivez la requête qui cherche, semaine d'assemblage par semaine d'assemblage, la part des motoneiges connectées de la ligne `LMN` ayant levé un code du circuit de refroidissement, entre la mi-septembre et la mi-novembre 2025. Donnez le résultat et dites ce que la forme de la courbe permet et ne permet pas.

Diagnostics

E1. La fenêtre qui masque la faute. Un collègue a lu la section 5, en retient qu'une fenêtre de reprise rattrape les retardataires, et l'écrit sur la date de la sortie plutôt que sur la date de réception : `where cle_date > (select max(cle_date) - interval 3 day from {{ this }})`. Il rejoue les passes du labo et retrouve les 201 542 lignes, sans une manquante. Dites pourquoi cela marche, dites ce que cela coûte par rapport au bon filtre, et dites dans quelles conditions cela cesse de marcher, avec le chiffre.

E2. L'accumulation rendue incrémentale. Un collègue voit que `fct_reclamations` se reconstruit en entier à chaque exécution et décide de l'améliorer : matérialisation `incremental`, `unique_key` sur `numero_reclamation`, filtre `where date_ouverture > (select max(cle_date_ouverture) from {{ this }})`. Les trois lots de 2026 passent, `dbt build` reste au vert, et la somme des montants tombe juste. Dites ce qui est faux, donnez le compte, et dites pourquoi aucun test du projet ne l'attrape.

E3. La fenêtre qui relit tout. Un collègue préfère `charge_le` à `recu_le`, en expliquant que l'instant du chargement est la seule horloge que le pipeline maîtrise. Il a raison sur le principe. Il rejoue le labo dans l'après-midi, charge les trois lots à quelques minutes d'intervalle, et trouve que l'incrémental ne va pas plus vite qu'un plein rebâti. Dites pourquoi, donnez le chiffre, et dites si sa préférence était bonne quand même.

© 2026 iAlexMG. Tous droits réservés.

E4. Le `--full-refresh` de confort. Un collègue ajoute `--full-refresh` à la tâche mensuelle, « par sécurité, pour être sûr que tout est juste ». La tâche passe au vert, les chiffres sont bons, et il tourne ainsi pendant un an. Dites ce que cette option rend inutile, ce qu'elle cache, et le jour où le problème paraîtra.

E5. La cible oubliée. Un collègue met le pipeline en production : une tâche planifiée qui lance `python -m pipeline raw` puis `dbt build --exclude snp_concessionnaire`. La tâche passe au vert chaque mois pendant trois mois. Personne ne se plaint, et le tableau de bord de la direction n'a pas bougé d'une ligne depuis trois mois. Dites pourquoi, et dites ce qui aurait dû le signaler.

Questions à développer

A. La largeur d'une fenêtre de reprise. Une fenêtre trop étroite laisse passer des trous, une fenêtre trop large coûte du temps et donne une fausse tranquillité. Expliquez comment vous choisiriez la largeur sur une source que vous ne connaissez pas encore, ce que vous mesureriez, et ce que vous écririez à côté du chiffre pour que votre successeur puisse le remettre en question.

B. Ce qu'un incrémental promet. Un modèle incrémental promet que le résultat est le même que si l'on avait tout reconstruit. Expliquez ce qui, dans le projet, rend cette promesse vérifiable plutôt que déclarée, dites à quelle fréquence vous la vérifieriez, et nommez le moment où elle se casse le plus souvent.

C. (avancé) Le verdict, trois mois plus tard. Au Cours 06, vous aviez répondu au directeur de la qualité que les 240 motoneiges de la cohorte ne se distinguaient pas des autres. Les données de 2026 disent maintenant 14,35 pour cent de réclamations de refroidissement contre 1,16. Rédigez ce que vous lui écrivez, dites ce que vous faites de votre réponse précédente, et dites ce que vous proposez comme suite.

Mises en situation

M1. Bornes de recharge de Kamouraska. Un réseau de bornes publiques publie une session de recharge par message, sur un courtier, avec l'heure de début, l'heure de fin, l'énergie livrée et l'identifiant de la borne. Les bornes des rangs perdent le réseau l'hiver et publient en rafale au dégel. On vous demande un entrepôt des sessions. Dites quel type de fait vous montez, sur quelle colonne vous filtrez, comment vous choisissez la fenêtre, et ce que vous demandez à l'équipe qui exploite le courtier.

M2. (avancé) Papeterie de La Tuque. La papeterie livre chaque nuit un export complet de ses ordres de fabrication, pas un delta : le fichier du jour contient tous les ordres ouverts et ceux fermés dans les trente derniers jours, chacun dans son état courant. On vous demande un fait des ordres de fabrication, et le volume interdit de tout reconstruire. Dites comment vous vous y prenez, ce que vous prenez comme clé, ce que vous faites des ordres qui disparaissent de l'export, et ce que vous promettez au directeur de l'usine.

M3. Coopérative de taxis de Val-d'Or. La coopérative a un entrepôt qu'un seul analyste fait tourner à la main, quand on le lui demande. Elle veut le mettre en production. Elle n'a ni équipe de données, ni ordonnanceur, ni environnement de développement. Dites ce que vous mettez en place en premier, ce que vous laissez pour plus tard, et la seule chose que vous refusez de livrer sans.

© 2026 iAlexMG. Tous droits réservés.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Comment lire ce corrigé. Il donne la réponse et le pourquoi, jamais une notion neuve : tout ce qu'il emploie a été enseigné dans la Théorie 07 ou pratiqué dans le Labo 07. Quand une réponse repose sur un geste du labo, le renvoi le nomme.

Questions courtes

1. Le chantier et le régime permanent. Ce qui est neuf, ce qui a changé, et ce qu'on fait après une panne. La troisième est la plus importante : un pipeline qu'on n'ose pas relancer est cassé, quel que soit le reste. Les deux premières décident de la forme du filtre, la troisième décide si le modèle est utilisable la nuit où quelque chose tombe. *Théorie : `Theorie_07_AE.md`, section 1.*

2. Les deux régimes. À la première exécution, la table n'existe pas et dbt la crée en entier, exactement comme une `table`. Ensuite, elle existe, et dbt n'exécute que ce que le filtre laisse passer, puis fusionne. Ce qui décide n'est ni une option ni une date : c'est la présence de la table. D'où le comportement qui surprend, une bascule de `table` vers `incremental` ne reconstruit rien, parce que la table était déjà là. *Théorie : `Theorie_07_AE.md`, section 3.*

3. Les trois pièces. `is_incremental()` vaut faux à la première exécution et vrai ensuite : elle ouvre le bloc qui porte le filtre. `{{ this }}` désigne la table qu'on est en train de bâtir, et c'est la seule façon de lui demander où elle s'est arrêtée. `unique_key` déclare la clé du grain : une ligne dont la clé existe déjà remplace la sienne au lieu de s'ajouter. *Théorie : `Theorie_07_AE.md`, section 3.*

4. Vitesse et justesse. Le filtre sert la vitesse, la clé sert la justesse. La faute grave est l'absence de clé : sans filtre, un modèle est seulement lent ; sans clé, il accumule des doublons dès la première relecture, et personne ne le remarque avant qu'un total soit faux. *Théorie : `Theorie_07_AE.md`, section 3.*

5. `{{ this }}`. Il désigne la relation que le modèle produit, celle qui existe déjà au moment où on la lit. Hors du bloc ouvert par `is_incremental()`, il désignerait une table absente à la première exécution, et la requête échouerait avant d'avoir rien créé. *Théorie : `Theorie_07_AE.md`, section 3.*

6. La stratégie. `delete+insert` supprime d'abord les lignes dont la clé revient dans le lot, puis insère le lot en entier. Elle suppose que `unique_key` est vraiment une clé du grain : si deux lignes du lot partagent la même valeur, la suppression en enlève une et l'insertion en remet deux, et le grain est perdu sans qu'aucune erreur ne soit levée. *Théorie : `Theorie_07_AE.md`, section 3.*

7. Quoi rendre incrémental. Ce qui est gros et qui ne fait que grossir. Chez Boréal, un seul fait le mérite aujourd'hui, `fct_sorties`, avec ses 201 542 lignes, et `fct_codes_sortie` le suit parce qu'il vient de la même source et du même rythme. `fct_ventes` (80 618), `fct_enregistrements` (28 118) et `fct_production` (29 782) sont trop petits pour que cela vaille la peine ; `fct_reclamations` (4 337) l'est aussi, et en plus ses lignes changent, ce qui l'exclut pour une seconde raison. *Théorie : `Theorie_07_AE.md`, sections 3 et 7.*

8. La perte du filtre naïf. Quarante-sept sorties, 2 558,2 kilomètres et 2 codes d'anomalie, sur les 2026-01-30 et 2026-01-31 pour la passe de février, les 2026-02-27 et 2026-02-28 pour celle de mars, soit 2, 22, 2 et 21. *Labo : `Labo_07_AE.md`, étape 8.*

© 2026 iAlexMG. Tous droits réservés.

9. Pourquoi elle n'est pas aléatoire. Parce que les retardataires d'une période sont, par construction, les sorties des tout derniers jours : ce sont elles qui n'ont pas eu le temps d'arriver avant la coupure. Un rapport mensuel regarde précisément ces jours-là, donc la perte frappe la ligne qu'on lit le plus, et elle est toujours dans le même sens, à la baisse. *Théorie : `Theorie_07_AE.md`, section 4.*

10. La cadence. Sur toute l'archive, la même faute perdrait 125 sorties au rythme mensuel et 2 559, pour 116 428,6 kilomètres, au rythme quotidien, qui est celui de la source. La règle : plus un pipeline tourne souvent, plus un filtre sur l'événement perd, parce qu'il crée une coupure à chaque passe et que chaque coupure a ses retardataires. *Théorie : `Theorie_07_AE.md`, section 4.*

11. La bonne colonne. Sur la date d'arrivée, ici `recu_le`. Ce n'est jamais celle que le métier lit, parce que la colonne de date d'un fait est par construction celle de l'événement, et qu'un événement et son arrivée sont deux instants différents. Cela impose de garder dans le fait une colonne qui ne sert à personne d'autre qu'au pipeline, et c'est une raison suffisante de l'y mettre. *Théorie : `Theorie_07_AE.md`, section 4.*

12. La fenêtre de reprise. Elle assure une propriété que rien n'impose à la source, qu'un message n'arrive jamais avec une date de réception antérieure à une date déjà vue. Elle coûte 2 199 et 2 772 lignes relues aux deux dernières passes, contre 29 843 et 12 918 neuves. Elle ne rapporte rien sur ce jeu parce que la propriété y tient : les passes avec fenêtre donnent exactement le même résultat que sans, et le dire vaut mieux que fabriquer une démonstration. *Théorie : `Theorie_07_AE.md`, section 5.*

13. La largeur. Du plus long retard mesuré sur la source, 96 heures. Trois jours est donc une marge serrée et sept une marge confortable. Une fenêtre de trente jours dit que personne n'a mesuré : elle donne la tranquillité sans le raisonnement, et elle coûte 11 624 lignes relues à chaque passe au lieu de 788. *Théorie : `Theorie_07_AE.md`, section 5.*

14. La propriété vérifiée. L'archive compte 933 jours et aucun n'a été livré deux fois. Et les 2 943 messages livrés en double le sont toujours le même jour, à une minute d'écart au plus. *Labo : `Labo_07_AE.md`, étape 3.*

15. La clé et la relecture. Sans `unique_key`, la fenêtre ajoute ses lignes au lieu de les remplacer, et le modèle devient pire que s'il n'y avait pas de fenêtre. Mesuré sur une seule passe : 202 330 lignes pour 201 542 messages distincts, soit les 788 lignes de la fenêtre comptées deux fois. *Labo : `Labo_07_AE.md`, étape 11.*

16. `--full-refresh`. On le lance après toute retouche d'un modèle incrémental ou de ce qu'il lit, et pour comparer un chargement par tranches à un chargement d'un bloc. Les deux commandes qui le mesurent sur un instantané disent la même chose : `dbt snapshot --full-refresh` est refusé par dbt 1.12.3, qui ne connaît pas l'option sur cette commande, et `dbt build --full-refresh` laisse les 205 versions et les 49 fermetures intactes. La phrase qui circule est fausse ici. *Théorie : `Theorie_07_AE.md`, section 6.*

17. Le contrôle qui compte. La comparaison avec la source, ou le plein rebâti : les deux disent la même chose. Un test de grain vérifie que les lignes présentes ne se dédoublent pas, un test de relation que leurs clés pointent quelque part ; ni l'un ni l'autre ne peut parler d'une ligne absente, et c'est précisément ce qu'un filtre incrémental fautif produit. *Théorie : `Theorie_07_AE.md`, section 6.*

18. Le fait d'accumulation. Parce que ses lignes changent après coup : les trois lots de 2026 réécrivent 154 des 3 348 réclamations déjà livrées, soit 4,6 pour cent. Une réconciliation sur l'argent ne verrait rien, et c'est le détail qui achève l'argument : la somme des montants de ces 3 348 dossiers vaut 1 459 608,53 dollars avant comme après. Le montant est fixé à la soumission ; ce qui bouge, ce sont les jalons. *Théorie : `Theorie_07_AE.md`, section 7.*

© 2026 iAlexMG. Tous droits réservés.

19. La dérive de schéma. Le micrologiciel 2.0, diffusé à partir du 1er mai 2025, renomme `temp_moteur_max` en `temperature_moteur_max` et ajoute `heures_moteur`. Sur l'archive complète, 87 374 messages portent l'ancien nom et 114 168 le nouveau, et aucun ne porte les deux. Les deux noms se réunissent au `STAGING`, en une expression, parce que la couche a pour seule responsabilité de rendre la source lisible et qu'aucun modèle en aval ne doit savoir qu'il y a eu deux versions. *Théorie : `Theorie_07_AE.md`, section 2.*

20. Le semi-structuré. Un objet imbriqué, `position`, qui se descend d'un cran à la fois avec `->` puis `->>`. Une liste, `codes`, qui ne devient pas une colonne mais se déplie en lignes dans son propre fait. Et un champ renommé, que le `coalesce` réunit. Les trois se traitent au `STAGING`, et rien en aval ne voit de JSON. *Théorie : `Theorie_07_AE.md`, section 2.*

21. Les deux horloges. `recu_le` est l'heure à laquelle l'abonné du courtier a reçu le message, `charge_le` l'instant où Data Engineering l'a écrit dans la `RAW`. C'est `recu_le` qui porte le filtre du fait, et le diagnostic E3 dit pourquoi l'autre, qui semble pourtant plus rigoureux, se comporte mal quand on rejoue. *Théorie : `Theorie_07_AE.md`, sections 2 et 5.*

22. Deux environnements. La cible, choisie à l'exécution par `--target`, et la macro `generate_schema_name` qui en tire le préfixe du schéma. Le code n'en sait rien : le même dépôt et les mêmes modèles servent les deux environnements, et c'est ce qui rend la production reproductible, puisque ce qui a été éprouvé en développement est exactement ce qui s'exécute. *Théorie : `Theorie_07_AE.md`, section 9.*

23. L'orchestration. `dbt build` fournit déjà le quoi et l'ordre : il parcourt le graphe, construit chaque modèle puis le teste, et saute tout ce qui descend d'un échec. L'ordonnanceur ajoute le quand, et prévient quand ça casse. Le reste appartient à Data Engineering, qui a monté Airflow à son Cours 06 ; ce qui revient à l'ingénieur analytique est le contrat de la tâche, qu'elle tourne après le chargement, qu'elle soit rejouable, et qu'un test en échec arrête ce qui vient après lui. Le modèle fautif, lui, est déjà écrit quand son test tombe : la tâche doit finir en échec pour que quelqu'un le voie. *Théorie : `Theorie_07_AE.md`, section 10.*

24. Le verdict et sa contre-épreuve. 14,35 pour cent de réclamations de refroidissement dans la cohorte exposée, 34 sur 237 véhicules vendus, contre 1,16 pour cent chez les autres motoneiges de l'année-modèle, 29 sur 2 498. Les trois codes qui ne séparent pas les groupes sont ce qui fait la différence entre une corrélation et un mécanisme : si la cohorte était simplement plus surveillée, ou plus roulée, ou mieux suivie, tous les codes monteraient. Seuls ceux du circuit de refroidissement montent, et c'est exactement ce qu'un collier mal serré prédit. *Labo : `Labo_07_AE.md`, étape 18.*

Questions pratiques

P1. Le journal des passes.

```sql
select date_trunc('month', recu_le)::date as mois_de_reception,
       count(*) as sorties,
       count(distinct numero_serie) as vehicules,
       round(sum(distance_km), 1) as km
from marts.fct_sorties
where recu_le >= date '2025-10-01'
group by 1 order by 1;
```


© 2026 iAlexMG. Tous droits réservés.

Mois de réception	Sorties	Véhicules	Kilomètres
2025-10	8 646	2 696	278 669,8
2025-11	7 271	2 152	229 116,5
2025-12	9 240	3 940	418 792,2
2026-01	15 856	4 663	771 646,9
2026-02	16 925	4 740	844 566,5
2026-03	12 918	4 690	557 869,5

La saison se lit sans qu'on la cherche : janvier et février sont les deux plus gros mois, en sorties comme en kilomètres, et mars retombe d'un tiers. C'est un parc où la motoneige domine, et le creux de novembre, 7 271 sorties pour 2 152 véhicules, est l'entre-deux-saisons. La dernière ligne rappelle aussi que l'archive s'arrête au 31 mars : un mois de mars à 12 918 sorties n'est pas un mois faible, c'est le dernier mois livré. *Labo : `Labo_07_AE.md`, étape 7.*

P2. Le prix de la fenêtre.

```
with borne as (select max(recu_le) as repere from marts.fct_sorties)
select j as fenetre_jours,
       (select count(*) from marts.fct_sorties, borne
        where recu_le > repere - (j || ' day')::interval) as lignes_relues
from (select unnest([1, 3, 7, 30]) as j);
```

Fenêtre	Lignes relues
1 jour	133
3 jours	788
7 jours	1 883
30 jours	11 624

La progression n'est pas linéaire, et c'est ce qui rend le choix facile : passer de trois à sept jours double la relecture et couvre presque deux fois le plus long retard observé, 96 heures. Trente jours la multiplie par quinze sans rien couvrir de plus, puisque aucune source ne livre avec un mois de retard ici. La bonne largeur est la plus petite qui dépasse confortablement le retard mesuré, et le chiffre du retard doit être écrit à côté du modèle pour que ce raisonnement se refasse. *Théorie : `Theorie_07_AE.md`, section 5.*

P3. Le contrôle en test. Dans `tests/assert_sorties_completes.sql` :

© 2026 iAlexMG. Tous droits réservés.

```sql
select v.numero_message, v.recu_le
from {{ ref('stg_sorties') }} as v
where not exists (
    select 1 from {{ ref('fct_sorties') }} as f
    where f.numero_message = v.numero_message
)
```

Il rend 0 ligne. Ce qu'il coûte : il lit toute la vue de `STAGING` à chaque construction, donc il refait la déduplication des 201 542 messages, ce que la passe incrémentale cherchait justement à éviter. Sur ce volume, cela se paie en dixièmes de seconde et ne se discute pas. On le garde parce qu'il est le seul contrôle qui parle des lignes absentes : le grain, les relations et la réconciliation regardent tous ce qui est là. Le jour où il coûtera trop cher, on le rendra moins fréquent, pas moins vrai. *Labo : `Labo_07_AE.md`, étapes 8 et 14.*

P4. Le retard, par tranche.

```sql
with r as (select recu_le - fin as ecart from staging.stg_sorties)
select case when ecart <= interval 1 hour then 'une heure ou moins'
            when ecart <= interval 24 hour then 'de 1 a 24 heures'
            when ecart <= interval 72 hour then 'de 24 a 72 heures'
            else 'plus de 72 heures' end as tranche,
       count(*) as sorties
from r group by 1 order by 2 desc;
```

Tranche	Sorties
Une heure ou moins	193 209
De 1 à 24 heures	7 910
De 24 à 72 heures	382
Plus de 72 heures	41

Les trois dernières tranches font 8 333, le chiffre de la théorie. La queue est courte et elle s'arrête net : 41 sorties au-delà de trois jours, et aucune au-delà de quatre. Si une cinquième tranche apparaissait, plus de 96 heures, il faudrait deux choses avant de toucher au modèle : demander à l'équipe du courtier ce qui a changé, et élargir la fenêtre de reprise au nouveau maximum. Le second geste sans le premier soigne le symptôme. *Théorie : `Theorie_07_AE.md`, sections 4 et 5.*

P5. Un fait sans horloge d'arrivée. La colonne est `_charge_le`, que la `RAW` porte sur chacune de ses tables et qui dit quand Data Engineering a écrit la ligne. Il faut l'exposer au `STAGING`, comme `stg_sorties` le fait, en ajoutant `_charge_le as charge_le` à `stg_enregistrements`. Le bloc :

```sql
{{
    config(
        materialized = 'incremental',
        unique_key = 'numero_enregistrement',
```


© 2026 iAlexMG. Tous droits réservés.

```
        incremental_strategy = 'delete+insert'
    )
}}
...
{% if is_incremental() %}
where e.charge_le > (
    select max(charge_le) - interval '{{ var("fenetre_de_reprise_jours", 3) }}' day
    from {{ this }}
)
{% endif %}
```

Le lot de livraison ne ferait pas l'affaire, et pourtant il en a l'air : mesuré, aucun des 28 118 enregistrements n'arrive dans un lot postérieur à son propre mois, sur 51 lots. Le lot est donc une horloge fidèle sur cette source. Mais un lot est un nom, et un nom n'a pas d'ordre garanti : la même livraison a produit `2023-03_bis`, qui se trie juste après `2023-03` et qui est arrivé bien après `2023-05`. Un repère posé sur le nom du lot l'aurait sauté en entier. `_charge_le` n'a pas ce défaut, parce que c'est le pipeline qui l'écrit au moment où il écrit la ligne. *Théorie : `Theorie_07_AE.md`, sections 4 et 5.*

P6. Les codes par gamme.

```
select p.gamme,
       count(distinct c.cle_vehicule) as vehicules_avec_code,
       count(*) as codes_leves
from marts.fct_codes_sortie as c
inner join marts.dim_vehicule as v on v.cle_vehicule = c.cle_vehicule
inner join marts.dim_produit as p on p.code_article = v.code_article
group by 1 order by 3 desc;
```

Gamme	Véhicules avec un code	Codes levés
Hors-route	485	560
Motoneige	444	549
Motomarine	46	46

Ce qui frappe est la dernière ligne, et elle ne se lit pas telle quelle : il faut le dénominateur. Les motomarines connectées sont 1 386, les motoneiges 3 697 et les hors-route 2 561, ce qui donne 3,3 pour cent de motomarines ayant levé un code, contre 12,0 et 18,9. L'écart est réel et il s'explique : une motomarine roule une courte saison, sur l'eau, moins d'heures, et les cinq codes du guide portent tous sur des organes que la chaleur, la charge et la transmission fatiguent. Deuxième chose : chaque motomarine concernée n'a levé qu'un seul code, 46 pour 46, là où les deux autres gammes en cumulent plusieurs par véhicule. *Labo : `Labo_07_AE.md`, étapes 16 et 17.*

P7. Le parc connecté.

© 2026 iAlexMG. Tous droits réservés.

```sql
select v.annee_modele,
       count(distinct v.cle_vehicule) as assembles,
       count(distinct e.cle_vehicule) as vendus_au_detail,
       count(distinct s.cle_vehicule) as connectes
from marts.dim_vehicule as v
left join marts.fct_enregistrements as e on e.cle_vehicule = v.cle_vehicule
left join marts.fct_sorties as s on s.cle_vehicule = v.cle_vehicule
group by 1 order by 1;
```

Année-modèle	Assemblés	Vendus au détail	Connectés
2022	3 397	3 397	0
2023	6 816	6 804	0
2024	7 108	7 108	3 143
2025	7 289	7 165	3 090
2026	5 172	3 644	1 411

La précaution est double, et elle vaut pour toute analyse fondée sur la télémétrie. Rien n'existe avant l'année-modèle 2024 : le module n'équipait pas les véhicules antérieurs, et une comparaison entre années confondrait l'effet du module et celui de l'année. Et même là où il existe, moins d'un véhicule vendu sur deux publie, parce que le client doit y consentir. Un taux calculé sur les véhicules connectés n'est donc pas un taux du parc, et il faut le dire chaque fois qu'on le publie. *Labo : `Labo_07_AE.md`, étape 18.*

P8. La fenêtre retrouvée.

```sql
with connectees as (
    select p.cle_vehicule, p.cle_date_assemblage as jour_assemblage
    from marts.fct_production as p
    inner join marts.dim_actif as a on a.cle_actif = p.cle_actif
    where a.actif_id = 'LMN'
      and p.cle_date_assemblage between date '2025-09-15' and date '2025-11-15'
      and exists (select 1 from marts.fct_sorties s where s.cle_vehicule = p.cle_vehicule)
),
refroidissement as (
    select distinct c.cle_vehicule
    from marts.fct_codes_sortie as c
    inner join marts.dim_anomalie as a on a.code_anomalie = c.code_anomalie
    where a.systeme = 'refroidissement'
)
select date_trunc('week', v.jour_assemblage)::date as semaine_d_assemblage,
```


© 2026 iAlexMG. Tous droits réservés.

```
        count(*) as connectees,
        count(*) filter (where r.cle_vehicule is not null) as avec_un_code,
        round(100.0 * count(*) filter (where r.cle_vehicule is not null) / count(*), 1) as taux
from connectees as v
left join refroidissement as r on r.cle_vehicule = v.cle_vehicule
group by 1 order by 1;
```

Semaine d’assemblage	Connectées	Avec un code	Taux
2025-09-15	50	3	6,0 %
2025-09-22	43	4	9,3 %
2025-09-29	41	0	0,0 %
2025-10-06	48	0	0,0 %
2025-10-13	45	0	0,0 %
2025-10-20	48	8	16,7 %
2025-10-27	48	11	22,9 %
2025-11-03	47	1	2,1 %
2025-11-10	52	1	1,9 %

Ce que la courbe permet : elle désigne les deux semaines du 20 et du 27 octobre sans qu’on lui ait donné une seule date, après trois semaines à zéro et avant deux semaines à moins de deux pour cent. Un analyste qui n’aurait jamais entendu parler du compresseur d’air poserait la question au bon endroit, et c’est exactement le service qu’un entrepôt doit rendre.

Ce qu’elle ne permet pas : conclure. Les deux semaines de la mi-septembre, à 6,0 et 9,3 pour cent, ne sont pas nulles, donc le signal n’est pas une marche d’escalier propre. Et chaque point repose sur une quarantaine de véhicules : trois cas de plus ou de moins déplacent un point de sept points de pourcentage. La courbe est une piste, elle n’est pas une mesure, et la confondre avec une mesure serait d’autant plus tentant qu’on connaît déjà la réponse. *Labo : `Labo_07_AE.md`, étape 18.*

Diagnostics

E1. La fenêtre qui masque la faute. Cela marche, et il faut commencer par le reconnaître : le rejeu des passes rend les 201 542 lignes, sans une manquante. La raison n’a rien à voir avec la conception. Au moment de la passe de février, le repère est la dernière date de sortie déjà chargée, le 31 janvier ; trois jours en

© 2026 iAlexMG. Tous droits réservés.

arrière, cela remonte au 28, et les retardataires perdus par le filtre strict sont ceux des 30 et 31. Ils repassent. La fenêtre marche parce que la cadence mensuelle est bien plus large que le retard maximal de la source.

Ce que cela coûte par rapport au bon filtre : rien, ou presque. Mesuré, la passe de février relit 32 034 lignes par la date de sortie contre 32 042 par la date de réception, et celle de mars 15 667 contre 15 680. Les deux fenêtres couvrent le même terrain.

Quand cela cesse de marcher : dès que la cadence se resserre. Au rythme quotidien, qui est celui de l'archive, une fenêtre de trois jours sur la date de sortie laisse encore passer 33 sorties, contre 2 559 sans fenêtre du tout ; il en faut sept pour n'en perdre aucune. Le filtre sur l'arrivée, lui, ne dépend pas de la cadence, et c'est la seule raison qui compte. Une assurance qui répare un défaut de conception est le meilleur moyen de ne jamais le voir. *Théorie : `Theorie_07_AE.md`, sections 4 et 5.*

E2. L'accumulation rendue incrémentale. Ce qui est faux : 154 des 3 348 réclamations déjà écrites gardent leur état de décembre 2025, alors que les lots de 2026 leur ont apporté une décision ou un paiement. Quarante-huit d'entre elles sont encore déclarées `SOUMISE` dans le fait alors qu'elles ne le sont plus dans la source. Le filtre sur la date d'ouverture ne les revoit jamais, puisque leur date d'ouverture est ancienne et que le repère ne fait que monter.

Pourquoi aucun test ne l'attrape : parce que tous les tests du projet regardent ce qui est là. Le grain tient, les clés pointent quelque part, les valeurs sont dans les listes acceptées, et les jalons vides sont légitimes par construction. Quant à la réconciliation, elle ne verrait rien même si on en écrivait une : la somme des montants de ces 3 348 dossiers vaut 1 459 608,53 dollars avant comme après, parce que le montant est fixé à la soumission et que ce sont les jalons qui bougent. Une erreur qui ne touche pas l'argent est une erreur qu'aucun contrôle d'argent ne trouve. *Théorie : `Theorie_07_AE.md`, section 7.*

E3. La fenêtre qui relit tout. Parce qu'il a tout chargé le même après-midi. `charge_le` est l'heure du chargement, pas celle de l'événement : les trois lots portent la même journée, donc une fenêtre de trois jours sur cette colonne couvre les trois lots et tout ce qui les précède. Mesuré, l'archive entière tient dans la fenêtre : un seul jour de chargement, 201 542 lignes relues. La passe incrémentale refait alors exactement le travail d'un plein rebâti, et il a raison de ne voir aucune différence de temps.

Sa préférence était bonne quand même, et c'est ce qui rend le cas intéressant. `charge_le` est bien la seule horloge que le pipeline maîtrise, et sur une source qui ne fournirait aucune heure d'arrivée, ce serait le bon choix. Ce qui est en cause n'est pas la colonne, c'est le rejeu : en production, les trois lots arrivent à un mois d'intervalle et la fenêtre de trois jours ne couvre que le dernier. La leçon est qu'un rejeu accéléré ne mesure pas ce qu'un pipeline fait en régime permanent, et qu'il faut savoir laquelle des deux situations on est en train d'observer. *Théorie : `Theorie_07_AE.md`, sections 2 et 5.*

E4. Le `--full-refresh` de confort. Ce que l'option rend inutile : l'incrémental lui-même. Le filtre n'est jamais évalué, la clé ne sert jamais, et le modèle se recalcule en entier tous les mois. Le gain de vitesse disparaît, ce qui n'est pas grave ici, cinq secondes contre cinq secondes.

Ce qu'elle cache est autrement plus sérieux : elle rend le résultat juste quel que soit le filtre. Un filtre sur la date de l'événement, un repère mal choisi, une clé absente, rien de tout cela ne se voit tant que chaque passe repart de zéro. Le projet a l'air de tourner en incrémental et tourne en réalité en plein rebâti, et personne ne le sait.

Le jour où le problème paraîtra est celui où quelqu'un retirera l'option, parce que la table sera devenue trop grosse pour se refaire dans la fenêtre de la nuit. Ce jour-là, le filtre s'appliquera pour la première fois, sur un modèle que personne n'a jamais éprouvé, et le défaut arrivera en même temps que la pression de le

© 2026 iAlexMG. Tous droits réservés.

corriger vite. La parade est celle de l'étape 14 : on compare les deux chemins de temps en temps, on ne fait pas tourner le mauvais en permanence. *Théorie : `Theorie_07_AE.md`, section 6.*

E5. La cible oubliée. Parce que la cible par défaut est `dev`, et que la macro `generate_schema_name` y préfixe tous les schémas. La tâche construit donc consciencieusement `dev_staging`, `dev_marts` et `dev_presentation`, treize, quinze et un objets, pendant que `staging`, `marts` et `presentation` gardent l'état du jour où quelqu'un a lancé la commande à la main pour la dernière fois. Tout est au vert parce que tout est vrai : les modèles se construisent, les tests passent, simplement pas là où on croit.

Ce qui aurait dû le signaler n'est pas un test, c'est un contrôle de fraîcheur : une source ou une table de production dont la date de chargement la plus récente n'avance plus est le seul signal qui distingue un pipeline qui tourne d'un pipeline qui tourne à côté. C'est le sujet du Cours 08, et c'est la raison pour laquelle un tableau de bord qui n'a pas bougé depuis trois mois doit inquiéter avant qu'un utilisateur s'en plaigne. *Théorie : `Theorie_07_AE.md`, section 9.*

Questions à développer

A. La largeur d'une fenêtre de reprise. Sur une source inconnue, on ne devine pas la largeur, on la mesure, et la mesure est la même que celle du labo : la distribution de l'écart entre l'instant de l'événement et son arrivée. Il faut trois chiffres. La part qui arrive à l'heure, pour savoir si le phénomène est marginal ou structurel. Le maximum observé, qui donne le plancher de la fenêtre. Et la forme de la queue, parce qu'une queue courte qui s'arrête net, comme les 41 sorties au-delà de trois jours chez Boréal, ne se traite pas comme une queue qui traîne.

À cela s'ajoute une question qu'aucune donnée ne répond, et qu'il faut poser à l'équipe qui exploite la source : dans quelles circonstances une livraison peut-elle être rejouée ou corrigée. Un historique sans incident ne prouve rien sur ce que le système a le droit de faire ; il prouve seulement qu'il ne l'a pas encore fait.

Ce qu'on écrit à côté du chiffre est ce qui rend la décision révisable : la date de la mesure, le maximum observé, la volumétrie relue à cette largeur, et la phrase qui dit ce qu'il faudrait constater pour l'élargir. Un successeur qui trouve « fenêtre de trois jours » sans rien d'autre n'a que deux options, garder par superstition ou changer au hasard. Un successeur qui trouve « trois jours, retard maximal mesuré 96 heures le 2026-03-31, 788 lignes relues par passe, à revoir si une sortie apparaît au-delà de 96 heures » peut décider. *Théorie : `Theorie_07_AE.md`, section 5.*

B. Ce qu'un incrémental promet. Ce qui rend la promesse vérifiable est qu'il existe un second chemin pour obtenir le même résultat, et qu'il est bon marché : `--full-refresh` reconstruit tout depuis la `RAW`, et la comparaison est une soustraction. C'est une chance qu'on n'a pas partout, et c'est une raison de plus de garder le `STAGING` en vues et la `RAW` intacte : le jour où le plein rebâti devient impossible, la promesse devient invérifiable et il ne reste que la confiance.

La fréquence se décide sur le coût. Tant qu'un plein rebâti tient en quelques secondes, on le fait à chaque changement du modèle ou de ce qu'il lit, et on peut même le faire au hasard une fois par semaine. Quand il tient en une heure, on le fait sur un échantillon, ou sur la dernière tranche, et on garde le contrôle complet pour les changements de logique.

Le moment où la promesse se casse le plus souvent n'est pas l'écriture du modèle, c'est la retouche de ce qu'il lit. On corrige une règle du `STAGING`, on vérifie que la vue est juste, et on oublie que le fait incrémental porte encore l'ancienne règle sur tout son historique. Rien ne le signale : les lignes anciennes sont là, elles ont l'air normales, et seules les lignes neuves suivent la nouvelle règle. C'est le cas qui justifie à lui seul de relancer un plein rebâti après toute modification en amont, et de le dire dans la revue de code plutôt que de compter sur la mémoire. *Théorie : `Theorie_07_AE.md`, sections 3 et 6.*

© 2026 iAlexMG. Tous droits réservés.

C. Le verdict, trois mois plus tard. Ce que j'écris au directeur de la qualité tient en quatre points, dans cet ordre.

Le chiffre d'abord, sans préambule : sur les 237 motoneiges de la cohorte vendues au détail, 34 ont une réclamation de garantie au circuit de refroidissement, soit 14,35 pour cent, contre 1,16 pour cent chez les 2 498 autres motoneiges de la même année-modèle. C'est douze fois plus.

La contre-épreuve ensuite, parce que c'est elle qui distingue ce message d'une coïncidence. Les véhicules connectés de la cohorte lèvent les deux codes du circuit de refroidissement dix fois plus souvent que le reste du parc, et les trois autres codes au même rythme. Si la cohorte était simplement plus surveillée ou plus roulée, tous les codes monteraient.

Ce que je fais de ma réponse précédente, ensuite, et je le dis explicitement plutôt que de laisser le directeur le découvrir : en décembre, je lui ai écrit que rien ne distinguait cette cohorte, sur 5 réclamations pour 201 véhicules vendus. Cette réponse était juste avec les données de décembre et elle est fausse aujourd'hui, non parce que le raisonnement était mauvais mais parce que les défauts de serrage mettent des mois à paraître et que l'entrepôt s'arrêtait au 31 décembre. Je préfère qu'il retienne cela de moi plutôt que d'un tiers : une réponse datée doit être rouverte quand les données avancent, et c'est le travail de celui qui l'a donnée, pas de celui qui l'a reçue.

Ce que je propose enfin. La cohorte est identifiée par ses numéros de série, `MN26001238` à `MN26001477`, ce qui rend un rappel ciblé possible sans toucher au reste de la production. Et je propose de transformer la requête en surveillance : le taux de réclamation par semaine d'assemblage et par ligne, recalculé à chaque lot, avec un seuil qui alerte. Le cas qu'on vient de vivre aura alors servi à deux choses au lieu d'une.

Ce que je ne dis surtout pas : que le compresseur est la cause. Rien dans l'entrepôt ne l'établit, d'autres choses ont pu changer à l'usine dans ces deux semaines, et la question relève de l'ingénierie, pas de moi. Je ne dis pas non plus « les données prouvent » : elles montrent une cohorte qui tombe en panne douze fois plus souvent que ses voisines sur un organe précis, ce qui est une raison d'agir et n'est pas une preuve. *Théorie : `Theorie_07_AE.md`, section 11.*

Mises en situation

M1. Bornes de recharge de Kamouraska. Un fait de transaction, une ligne par session de recharge, au grain de la session : c'est un événement indépendant, avec un début, une fin et une mesure qu'on additionne, l'énergie livrée. Rien ici ne demande une autre forme.

Le filtre porte sur l'heure de réception du message par le courtier, jamais sur l'heure de fin de la session, et les bornes des rangs sont exactement la raison : une borne qui perd le réseau tout l'hiver publie au dégel des sessions vieilles de plusieurs mois. Un filtre sur l'événement les perdrait toutes, et c'est le pire cas possible, parce que les sessions perdues seraient systématiquement celles des bornes les moins bien couvertes, donc celles dont on a le plus besoin pour décider où investir.

La fenêtre se choisit sur la distribution de l'écart entre la fin de la session et sa réception, mesurée sur ce que le courtier a déjà archivé, et pas sur une valeur ronde. Si la queue va jusqu'à plusieurs mois, la fenêtre ne peut pas la couvrir, et c'est alors le filtre sur la réception qui fait tout le travail : la fenêtre reste étroite et sert seulement d'assurance.

© 2026 iAlexMG. Tous droits réservés.

Ce que je demande à l'équipe du courtier, c'est ce qu'aucune donnée ne dira : est-ce qu'une archive déjà écrite peut être réécrite, et dans quelles circonstances. De la réponse dépend tout le reste. Je demande aussi si l'identifiant du message est stable entre deux publications de la même session, puisque c'est lui qui deviendra la clé unique.

M2. Papeterie de La Tuque. Un export complet plutôt qu'un delta change la nature du problème : la source ne dit pas ce qui a changé, elle dit ce qui est. C'est le cas du `STAGING` de Boréal, en plus lourd, et la parade est la même en deux temps.

Au `STAGING`, on tire l'état courant : une vue qui garde, pour chaque ordre, la version du dernier export où il paraît. Le fait, lui, est incrémental avec le numéro d'ordre pour clé unique et un filtre sur l'instant de chargement de l'export, pas sur une date de l'ordre : un ordre vieux de trois semaines qui change d'état aujourd'hui arrive dans l'export d'aujourd'hui, et c'est tout ce qui compte. Avec `delete+insert` et cette clé, chaque ordre est remplacé par sa version courante, et la fenêtre de trente jours de l'export tombe naturellement dans la fenêtre de reprise, qu'on alignera sur elle.

Les ordres qui disparaissent de l'export sont la vraie difficulté, et il faut la nommer : un ordre fermé depuis plus de trente jours cesse d'être livré, et un fait incrémental ne le supprimera jamais tout seul, ce qui est ici le comportement voulu. On garde donc la dernière version connue, et on ajoute une colonne qui dit la date du dernier export où l'ordre est apparu. Ce qu'il ne faut surtout pas faire est de supprimer du fait ce qui n'est plus dans l'export : on effacerait l'historique.

Ce que je promets au directeur : que le fait reflète l'état des ordres au dernier export, que rien ne se perd quand un ordre sort de la fenêtre des trente jours, et que je peux reconstruire le fait entier depuis les exports conservés si la logique change. Ce que je ne promets pas, c'est l'histoire des états intermédiaires : elle n'existe que si l'on garde les exports ou si l'on monte un snapshot, et cela se décide avant, pas après.

M3. Coopérative de taxis de Val-d'Or. Ce que je mets en place en premier n'est pas l'ordonnanceur, c'est la reproductibilité : une seule commande qui fait tout, `dbt build`, et qui se relance sans rien casser. Tant qu'un pipeline demande à quelqu'un de se rappeler l'ordre des gestes, le planifier ne fait que rendre l'oubli automatique.

Ensuite, et seulement ensuite, la séparation des environnements. Elle coûte une cible de plus dans le fichier de connexion et six lignes de macro, et elle évite le seul incident que cette coopérative ne pardonnera pas : un modèle à moitié réécrit qui atterrit dans ce que la direction consulte.

L'ordonnanceur peut attendre. Le planificateur du système suffit à lancer une commande à heure fixe, et il n'y a aucune raison d'installer Airflow pour un graphe qui tient en une commande. Ce que je laisse aussi pour plus tard : l'intégration continue, la documentation publiée, et tout ce qui suppose une équipe.

La seule chose que je refuse de livrer sans est un contrôle qui parle quand la chaîne s'arrête. Un pipeline solitaire qui échoue en silence est pire que pas de pipeline : la coopérative continuera de lire un tableau de bord qui a l'air normal et qui a trois semaines de retard, et elle décidera dessus. Un courriel quand la commande sort en erreur, et une date de dernière mise à jour affichée sur le tableau de bord lui-même : ces deux-là ne sont pas négociables, et ils tiennent en une après-midi.

© 2026 iAlexMG. Tous droits réservés.